

Il semble impossible de diminuer les indices CIP et CE ?

June 30, 2026

1 Ajouter un nouvel attribut ?

J'appelle attribut (feature) une liste de valeurs, une pour chaque pixel, qui peut être utilisée pour la classification non-supervisée.

Pour l'expérience 2, quand on rajoute un nouvel attribut, on pourrait croire qu'il s'agit d'une information supplémentaire et que même si elle n'est pas pertinente, elle ne va pas diminuer la performance. S'il était facile de mesurer précisément l'information, c'est-à-dire ici la pertinence de cet attribut vis-à-vis de l'objectif poursuivi, on pourrait imaginer des algorithmes qui atténuent l'importance de cet attribut de façon à ce que la performance ne diminue pas. Mais en gros ce qui se passe est plutôt que cet attribut a une influence sur la classification prédite et la pertinence de l'influence est plus ou moins mesurée, mais plus on mesure cette pertinence moins on a d'informations utilisables.

Le deuxième aspect que je voudrai discuter sur ce nouvel attribut est qu'il tend à ce que les classes forment des ensembles de pixels qui spatialement ressemble à un disque, en particulier une classe correspondant à plusieurs formes spatialement distinctes est pénalisée au sens où cet attribut n'aura alors pas des valeurs similaires. Il se trouve que le ground truth contient des classes ayant plusieurs formes spatialement distinctes. En revanche CE et CIP s'adaptent tout à fait à des classes ayant plusieurs formes spatialement distinctes.

2 Critique des attributs actuels

Il y a un point que je n'ai pas vérifié. Ce sont les tailles des classes que la minimisation du CE et du CIP favorisent. Je me dis que le CE a tendance à privilégier des classes de tailles très inégales, par exemple une classe a presque tous les pixels et les autres très peu de pixels. Cela ne semble pas être le cas dans les résultats de classifications montrés.

En Machine Learning, on utilise un facteur d'imbalance souvent noté I , c'est le nombre de pixels de la classe ayant le plus de pixels divisé par la classe ayant le moins de pixels.

On peut adapter les indicateurs à ce problème en changeant leur définition.

$$\text{taille du contour de } A \Rightarrow \min \frac{\text{taille du contour dans } A}{\sqrt{\text{taille de } A}}, \frac{\text{taille du contour extérieur à } A}{\sqrt{\text{taille extérieur à } A}} \quad (1)$$

$$\text{Nombre de points isolés de } A \Rightarrow \min \frac{\text{nombre de points isolés dans } A}{\sqrt{\text{taille de } A}}, \frac{\text{nombre de points isolés extérieur à } A}{\sqrt{\text{taille extérieur à } A}} \quad (2)$$

3 Tentatives pour faire fonctionner l'algorithme

Pour faire des expériences numériquement plus coûteuses, j'utilise `tmux` sur l'ordinateur à distance. Cela me permet de lancer une simulation sur l'ordinateur distant, arrêter la connexion avec l'ordinateur distant et ensuite me reconnecter pour voir les résultats de la simulation.

L'heuristique du recuit simulé, comme toute heuristique, suppose un comportement déterministe de la fonction à minimiser. Je veux dire que pour chaque ensemble de valeurs du vecteur `weights` la valeur obtenue pour le score doit être unique. A priori l'algorithme `kmeans` est souvent implémenté avec un aspect aléatoire et dans votre implémentation, cet aspect est contrôlé grâce à `seed`. Sauf qu'il se trouve que la valeur de `seed` dépend de `t` qui compte le nombre d'itérations. En fixant `seed`, j'ai observé qu'on obtient un algorithme qui est déterministe. Et au début l'heuristique fonctionne bien.

Dans la proposition de modification de l'heuristique, j'avais mis un paramètre noté dans l'implémentation `w_direction` qui pondère entre l'exploration d'une direction particulière qui a fonctionné et l'exploration d'une autre direction. Il est intéressant de fixer ce paramètre à 0.999 pour vraiment explorer cette direction, mais aussi de ramener ce paramètre à 0 au bout d'un certain nombre d'itérations pour aller ailleurs, dans l'hypothèse où l'échec est lié à minimum local.

En fait les résultats expérimentaux m'ont amené à penser que l'heuristique fonctionne bien au début, mais qu'ensuite elle n'obtient rien, tout simplement parce qu'il n'y a probablement pas de valeurs des coefficients `w` qui permettent d'améliorer les performances.

4 Critique de l'expérimentation

Dans l'expérimentation, c'est un seul vecteur `weights` qui est censé guider représenter les informations nécessaires pour que la classification non-supervisée marche bien pour les 16 classes à la fois.

Je me suis posé la question de savoir s'il y a une grande sensibilité vis-à-vis de la segmentation, c'est-à-dire si une petite modification de `weights` a un impact sur cette classification. La réponse est que si la modification est inférieur à l'ajout d'un bruit blanc gaussien d'écart-type de 5×10^{-5} , on a en général pas de modifications. Cela montre que le vecteur `weights` permet de coder des informations assez nombreuses.

Même si l'information requise peut éventuellement être codée dans `weights`, il reste que ce sont des informations potentiellement contradictoires, telle longueur d'onde est plus importante pour distinguer telle classe de telle classe mais pas de telle autre.

5 Nouvelle proposition

La première proposition consiste à utiliser l'algorithme `kmeans` 16 fois, la première fois pour récupérer la première classe, la deuxième fois uniquement sur les pixels qui ne sont pas dans cette classe et pour récupérer la deuxième classe etc... À chaque utilisation du `kmeans`, c'est un vecteur différent de `weights` qui est considéré. Il y a un petit défaut, c'est qu'il faut paramétrer le `kmeans` pour lui dire que la classe extraite doit être plus petite que la classe restante d'un facteur qui peut lui aussi être déterminé en fonction de `ce` et `cip`. Ici c'est l'architecture d'une échelle.

La deuxième proposition consiste à utiliser l'algorithme `kmeans` 4 fois suivant l'architecture d'un arbre. L'algorithme est utilisé ensuite au sein de chaque lots de pixels avec un `weights` différent.